EduGenie Project Documentation

Overview: EduGenie is an AI-powered personal tutor system designed for learning support. It provides features like structured explanations, Q&A assistance, automated quiz generation, textbook summarization, and adaptive learning recommendations.

1. System Architecture & Request Flow

The application runs on a **FastAPI Backend** which routes requests dynamically into various domain modules based on client requirements. The processed results are then rendered fluidly back to the web UI frontend framework.

Step / Component	Functionality / Route Description
1. Start Framework	User triggers interaction via the EduGenie unified dashboard UI interface.
2. Frontend Layer	Accepts parameters (queries, custom paragraphs, targeted educational topics).
3. FastAPI Router Backend	Intercepts parameters; conditionally transfers payload requests into distinct modular logic endpoints.
4. Modules & Endpoints	• Explanation Module (/explain) • Q&A Module (/qa) • Quiz Generator Module (/quiz) • Summarization Module (/summarize) • Learning Path Recommendation (/learn/recommendations)
5. Final Render Pipeline	Asynchronously converts payload responses back onto the client presentation template layers.

2. Repository Blueprint & Project Structure

The system directory tree breakdown is structured cleanly to support decoupled scaling of foundational modules:

```
EDUGENIE/ (Root Project Folder)
 ■■■ __pycache__/
 ■■■ static/
 ■    ■■■ style.css (Main interface stylesheets layout definitions)
 ■■■ templates/
 ■    ■■■ index.html (Central UI template view configuration document)
```

■■■ explanation_module.py (Preconfigured pipelines using LaMini-Flan-T5 models)

■■■ learning_path.py (Orchestrates structured timeline study recommendations)

■■■ main.py (The central entrypoint driving asynchronous server initializations)

■■■ qna.py (Direct wrapper logic communicating with AI API instances)

■■■ quiz_module.py (Generates validation assessments in valid nested format JSON output)

■■■ requirements.txt (Dependencies registry listing: fastapi, transformers, torch, etc.)

■■■ summary_module.py (Handles operational logic parameters filtering text contents)

3. Core Technical Configurations & Code Snippets

3.1 Core Router Server Entrypoint (main.py)

```
from fastapi import FastAPI, Request, Query
from fastapi.responses import JSONResponse
import uvicorn

@app.get('/qa')
async def answer_question(question: str = Query(...)):
    answer = answer_question_with_gemini(question)
    return {'answer': answer}

@app.post('/explain/')
async def explain_api(request: Request):
    data = await request.json()
    topic = data.get('topic')
    explanation = explain_topic(topic)
    return {'topic': topic, 'explanation': explanation}
```

3.2 Generative Assessment Engine Framework (quiz_module.py)

```
def generate_quiz(text: str) -> list:
    model = genai.GenerativeModel(model_name='models/gemini-1.5-pro')
    prompt = f"""You are a quiz generator. From the following passage, create 3 multiple-choice questions. Format your output as valid JSON..."""
    response = model.generate_content(prompt)
    cleaned_text = clean_json_block(response.text.strip())
    return cleaned_text
```

4. Production Uvicorn Engine Log Output Verification

A live console assertion runtime test validates system health and server bindings:

```
INFO:     Will watch for changes in directories: ['/Users/edugenie']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

```
INFO:     Started reloader process [17749] using StatReload
INFO:     Started server process [17751]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```